

Final Exam – Statistics Module

Master in Data Science and Big Data Analytics

Student: Adriano Fonzino

Date: November 3rd, 2020

Dataset's Description

Dataset name: Prostate_cancer.

Prostate's cancer is a family of very heterogeneous tumors of prostate gland, both on an histological and clinical level. In this dataset scientists analyzed the gene expression level of 6033 genes on samples collected from control and cancer tissues (prostate gland samples) by the use of **microarray**, a high-throughput molecular biology technique aimed to the study of a wide range of gene expression level (**RNA** transcribed by genes) in the same sample.

The dataset is a subset of data extracted by the one produced in 2002 by *Singh et al.* and already contains pre-processed data as gene expression levels.

Sample number: 102.

Sample were classified by physicians as *controls* (50 samples) and as *cancers* (52 sample).

Features number: 6034.

The dataset consisted of 102 rows and 6034 columns with the first column named as "Status", that is a factor with 2 levels "control" and "cancer", indicating the type of the collected and analyzed sampe by microarray's experiments. The other 6033 columns were gene expression's levels of transcript encoded by genes analyzed. These were double vectors of 102 dimensions.

Aim of the work

The aim of this work was to build and train, using the data offered by this microarrays' experiments, a **Logistic Regression classifier** able to distinguish among *control* and *cancer* prostate tissues, basing on the differences in **gene expression levels** and, eventually, to obtain a short list of genes that are important in order to classify samples. The procedure involved also a **screening** part with the objective to eliminate the less important genes, decreasing the complexity of the model and increasing its interpretability.

Methods

The "Status" feature was used as **response variable** y while the 102×6033 matrix of gene expression levels as putative **explanatory variables**.

Firstly, it was performed a **marginal t-test procedures** calculating **p-values** for every explanatory variable. The p-values calculation was done both by the use of both **theoretical** and **permutation** methods.

After that, it was done p-values correction by several **Multiple Testing Procedures** (MTP) in order to control the **type I error rate**. **MTPs** used were both Family-Wise Error Rate (**FWER**) and False Discovery Rate (**FDR**).

FWER procedures used were **Bonferroni** single-step, Sidak single-step (**SidakSS**), **Holm** step-down, Sidak step-down (**SidakSD**) and **Hochberg** step-up.

FDR procedures used were Benjamini & Hochberg step-up (**BH**) and Benjamini & Yekutieli step-up (**BY**).

In the end, only results obtained by **permutation** method were used. For the permutation method, was chosen **B = 5000** (number of permutation for each variable).

The workflow followed two parallel **paths**:

- 1) **without** the use MTP screening for the correction of p-values, using thus, only t-test significant's genes with $p < 0.05$ obtained by permutation method.
- 2) **with** the use of MTPs for the **correction of the p-values** obtained by permutation method.

Dataset was standardized and divided into **train** and **test sets**.

Resulting significant genes of the train set, selected for both paths, were used to perform a **features selection** through **Lasso Regularization** of Logistic Regression Model, tuning the **lambda** value by **Cross Validation** (CV) using Mean Squared Error (**MSE**) as measure of the *performance* to minimize.

The best Lasso model gave back a list of genes used to train a **Ridge Regression Model** with the **binomial** family. Also in this case, lambda minimum was obtained by CV technique on the **subset** of the train data (subset of genes selected by Lasso procedure).

Thus, two (one for each path) Final Ridge Regression models were trained on the whole training dataset, using the lambda minimum found by CV ever on the same subset of data.

Eventually, **evaluation of the performance** of the final models, in term of both **accuracy** and **confusion matrix**, were calculated on the test set. A probability of 0.5 was chosen as **threshold** to classify data points into *control* or *cancer* status.

Below copy of the commented **code** used to perform the analysis (a total of 2 files).

File 1: "Functions.R"

```
marginTTest <- function(X, y, NullDistr = c("theoretical", "permutation"),
                        alpha = 0.05, B = 500) {
  this.call <- match.call()
  NullDistr <- match.arg(NullDistr)
  if (NullDistr == "theoretical") {
    print("Testing using Theoretical Null Distribution")
    marginTTest.out <- pbapply(X, 2L, marginTTest.theodistr, y = y,
                              alpha = alpha)
  }
  else {
    print("Testing using Permutation Null Distribution")
    marginTTest.out <- pbapply(X, 2L, marginTTest.permdistr, y = y,
                              alpha = alpha, B = B)
  }
  marginTTest.out <- data.frame(Var = colnames(X), t(marginTTest.out))
  Decision <- rep("reject H0", times = ncol(X))
  Decision[marginTTest.out$p >= alpha] <- "do not reject H0"
  marginTTest.out$Decision <- Decision
  marginTTest.out <- as_tibble(marginTTest.out)
```

```

    out <- list(call = this.call, Summary = marginTTest.out)
    class(out) <- "marginTTest"
    out
  }

marginTTest.theodistr <- function(x, y, alpha) {
  out <- vector(mode = "numeric", length = 4L)
  names(out) <- c("TTest", "cv1", "cv2", "p-val")
  n <- length(y)
  X0 <- x[y == "control"]
  X1 <- x[y == "cancer"]
  out.ttest <- t.test(X0, X1)
  out["TTest"] <- out.ttest$statistic
  out["cv1"] <- qt(p = alpha / 2, df = n - 1)
  out["cv2"] <- qt(p = 1 - alpha / 2, df = n - 1)
  out["p-val"] <- out.ttest$p.value
  out
}

marginTTest.permdistr <- function(x, y, alpha, B) {
  out <- vector(mode = "numeric", length = 4L)
  names(out) <- c("TTest", "cv1", "cv2", "p-val")
  n <- length(y)
  X0 <- x[y == "control"]
  X1 <- x[y == "cancer"]
  out.ttest <- t.test(X0, X1)
  out["TTest"] <- out.ttest$statistic
  TStat.perm <- vector(mode = "numeric", length = B)
  for(b in seq_len(B)) {
    id <- sample(n)
    y.perm <- y[id]
    X0.perm <- x[y.perm == "control"]
    X1.perm <- x[y.perm == "cancer"]
    out.ttest.perm <- t.test(X0.perm, X1.perm)
    TStat.perm[b] <- out.ttest.perm$statistic
  }
  out[c("cv1", "cv2")] <- quantile(TStat.perm, probs = c(alpha / 2, 1 - alpha / 2))
  out["p-val"] <- mean(abs(TStat.perm) >= abs(out["TTest"]))
  out
}

# definition of methods' functions of the Class marginTTest

print.marginTTest <- function(x, ...) {
  cat("\nCall: ", paste(deparse(x$call), sep = "\n", collapse = "\n"), "\n\n", sep = "")
  cat("\nOutput della Procedura di Screening\n")
  cat("Numero Test Eseguiti = ", nrow(x$Summary), "\n\n")
  print(x$Summary, ...)
}

hist.marginTTest <- function(x, ...) {
  x$Summary %>% pull(p.val) -> p.val
  hist(p.val, ...)
}

```

File 2: "Final_Exam_Adriano_Fonzino.R"

```
#####  
# Code Final Exam Statistics – Master Data Science and Big Data Analytics || Student: Fonzino Adriano #####  
#####  
library("tibble")  
library("dplyr")  
source("FunzioniR.R")  
library(pbapply)  
library(multtest)  
library(glmnet)  
Prostata <- read.table(file = "Prostata.txt", header = TRUE)  
dim(Prostata)  
  
# extraction of the response variable  
y <- as.factor(Prostata$Status)  
  
# extraction of explanatory variables  
X <- as.matrix(Prostata[, -1])  
  
rm(Prostata)  
  
alpha <- 0.05  
  
#####  
##### SCREENING PROCEDURES (MTP) #####  
#####  
  
#####  
# THEORETICAL METHOD #  
#####  
  
out.ttest.theoretical <- marginTTest(X = X, y = y, NullDistr = "theoretical")  
out.ttest.theoretical  
hist(out.ttest.theoretical)  
  
out.ttest.theoretical$Summary %>% arrange(p.val) %>% pull(p.val) %>%  
  mt.rawp2adjp(proc = c("Bonferroni", "SidakSS", "Holm", "SidakSD", "Hochberg",  
    "BH", "BY")) -> adj_pval.theor  
  
Decision.theor <- ifelse(adj_pval.theor$adjp <= alpha, "reject Ho", "do not reject H0")  
  
out.ttest.theoretical$Summary %>% arrange(p.val) -> MTP.Results.theor  
for (i in c("Bonferroni", "SidakSS", "Holm", "SidakSD", "Hochberg",  
  "BH", "BY")) {  
  MTP.Results.theor[i] = as.tibble(Decision.theor)[i]  
}  
MTP.Results.theor  
  
MTP.Results.theor %>% select(Decision:BY) %>%  
  apply(2, table) -> MTP.Summary.05.theor  
MTP.Summary.05.theor  
  
tibble(Procedure = colnames(MTP.Summary.05.theor),  
  NoRej_H0 = MTP.Summary.05.theor[1L, ],  
  Rej_H0 = MTP.Summary.05.theor[2L, ]) -> MTP.Summary.05.theor  
MTP.Summary.05.theor  
write.table(MTP.Summary.05.theor, file = "immagini_tabelle\\1_Summary_theor.csv")
```

```
#####
# PERMUTATION METHOD #
#####

source("FunzioniR.R")

out.ttest.permutation <- marginTTest(X = X, y = y, NullDistr = "permutation",
                                     B = 5000)
out.ttest.permutation
hist(out.ttest.permutation)

out.ttest.permutation$Summary %>% arrange(p.val) %>% pull(p.val) %>%
  mt.rawp2adjp(proc = c("Bonferroni", "SidakSS", "Holm", "SidakSD", "Hochberg",
                        "BH", "BY")) -> adj_pval.perm

Decision.perm <- ifelse(adj_pval.perm$adjp <= alpha, "reject Ho", "do not reject H0")

out.ttest.permutation$Summary %>% arrange(p.val) -> MTP.Results.perm
for (i in c("Bonferroni", "SidakSS", "Holm", "SidakSD", "Hochberg",
            "BH", "BY")) {
  MTP.Results.perm[i] = as.tibble(Decision.perm)[i]
}
MTP.Results.perm

MTP.Results.perm %>% select(Decision:BY) %>%
  apply(2, table) -> MTP.Summary.05.perm
MTP.Summary.05.perm

tibble(Procedure = colnames(MTP.Summary.05.perm),
       NoRej_H0 = MTP.Summary.05.perm[1L, ],
       Rej_H0 = MTP.Summary.05.perm[2L, ]) -> MTP.Summary.05.perm
MTP.Summary.05.perm
write.table(MTP.Summary.05.perm, file = "immagini_tabelle\\2_Summary_perm.csv")

# Standardize data
X.scaled <- scale(X)

# divide into train and test sets
set.seed(1234)
id1 <- sample(which(y == "control"), size = 10)
id2 <- sample(which(y == "cancer"), size = 10)

Xtrain <- X.scaled[-c(id1,id2), ]
ytrain <- y[-c(id1, id2)]
dim(Xtrain)
length(ytrain)

Xtest <- X.scaled[c(id1,id2),]
ytest <- y[c(id1, id2)]
dim(Xtest)
length(ytest)

#####
# REGRESSION MODEL WITHOUT SCREENING #
#####

# selects significant genes without screening techniques
MTP.Results.perm %>% select(Var, Decision) %>%
  filter(Decision == "reject H0") %>% pull(Var) -> genes.sign05.perm
genes.sign05.perm
```

```

Xtrain.sign05.perm <- Xtrain[, genes.sign05.perm]
colnames(Xtrain.sign05.perm) == genes.sign05.perm

# p = 481 ==> p >> n ==> lasso selection!
obj.lasso <- glmnet(Xtrain.sign05.perm, ytrain, family = "binomial", alpha = 1)
names(obj.lasso)

plot(obj.lasso, xvar = "lambda", xlab = expression(log(lambda)),
     main = "Logistic Regression with lasso Penalization",
     lwd = 2, cex.lab = 1.6)

# select best lambda value by Cross Validation
out.lasso.cv <- cv.glmnet(x = Xtrain.sign05.perm, y = ytrain,
                        family = "binomial", type.measure = "mse")
out.lasso.cv

plot(out.lasso.cv, xvar = "lambda", xlab = expression(log(lambda)),
     main = "High-Dimensional Lasso Regression Model", lwd = 2, cex.lab = 1.6)

names(out.lasso.cv)
out.lasso.cv$lambda.min

# get lambda with the min. MSE
out.lasso.cv$lambda.min

# extract informations about the non zero coefficients selected from CV
idNonZero <- predict(out.lasso.cv, newx = XnoNA, type = "nonzero", s = "lambda.min")
lasso.coef <- predict(out.lasso.cv, newx = XnoNA, type = "coef", s = "lambda.min")
lasso.genes <- names(lasso.coef[lasso.coef[,1] != 0, ][-1])
lasso.genes

# select genes obtained from lasso feature selection
Xtrain.sign05.perm.lasso <- Xtrain.sign05.perm[, lasso.genes]
dim(Xtrain.sign05.perm.lasso)

# Ridge Regression for Predictive Model
out.ridge.cv <- cv.glmnet(x = Xtrain.sign05.perm.lasso, y = ytrain, alpha = 0,
                        family = "binomial", type.measure = "mse", keep=TRUE)
plot(out.ridge.cv, main = "Linear Regression Model with Ridge Penalty")
out.ridge.cv$lambda.min

coef(out.ridge.cv, s = "lambda.min")

final.ridge.model <- glmnet(x = Xtrain.sign05.perm.lasso, y = ytrain,
                          family = "binomial",
                          lambda = out.ridge.cv$lambda.min, alpha = 0)

yhat.ridge.cv <- predict(final.ridge.model, newx = Xtest[, lasso.genes],
                        type = "response", s = "lambda.min")
yhat.ridge.cv

yhat.ridge.cv.discrete <- ifelse(yhat.ridge.cv > 0.5, "control", "cancer")
View(cbind(as.factor(yhat.ridge.cv.discrete), ytest))

confusion.glmnet(final.ridge.model,
                 newx = Xtest[, lasso.genes], newy = ytest,
                 family = "binomial") -> confusion_05
confusion_05

```

```

write.table(confusion_05 , file = "immagini_tabelle\\3d_confusionM_05.csv")

#####
# REGRESSION MODEL WITH MTP SCREENING #
#####

MTP.Summary.05.perm

# Selecting genes from BH MTP screening
MTP.Results.perm %>% select(Var, BH) %>%
  filter(BH == "reject Ho") %>% pull(Var) -> genes.BH.perm
genes.BH.perm

Xtrain.BH.perm <- Xtrain[, genes.BH.perm]
colnames(Xtrain.BH.perm) == genes.BH.perm

# p = 35 ==> lasso selection!
obj.lasso.bh <- glmnet(Xtrain.BH.perm, ytrain, family = "binomial", alpha = 1)
names(obj.lasso.bh)

plot(obj.lasso.bh, xvar = "lambda", xlab = expression(log(lambda)),
     main = "Logistic Regression with lasso Penalization on BH genes",
     lwd = 2, cex.lab = 1.6)

# select best lambda value by Cross Validation
out.lasso.cv.bh <- cv.glmnet(x = Xtrain.BH.perm, y = ytrain,
                           family = "binomial", type.measure = "mse")
out.lasso.cv.bh

plot(out.lasso.cv.bh, xvar = "lambda", xlab = expression(log(lambda)),
     main = "High-Dimensional Lasso Regression Model", lwd = 2, cex.lab = 1.6)

names(out.lasso.cv.bh)
out.lasso.cv.bh$lambda.min

# get lambda with the min. MSE
out.lasso.cv.bh$lambda.min

# extract information about the non zero coefficients selected from CV
idNonZero.bh <- predict(out.lasso.cv.bh, newx = XnoNA, type = "nonzero", s = "lambda.min")
lasso.coef.bh <- predict(out.lasso.cv.bh, newx = XnoNA, type = "coef", s = "lambda.min")
lasso.genes.bh <- names(lasso.coef.bh[lasso.coef.bh[,1] != 0, ][-1])
lasso.genes.bh
length(lasso.genes.bh)

#We selected 27 genes from 35.
# select genes obtained from lasso feature selection
Xtrain.bh.perm.lasso <- Xtrain.BH.perm[, lasso.genes.bh]
dim(Xtrain.bh.perm.lasso)

# Ridge Regression for Predictive Model
out.ridge.cv.bh <- cv.glmnet(x = Xtrain.bh.perm.lasso, y = ytrain, alpha = 0,
                           family = "binomial", type.measure = "mse", keep=TRUE)
plot(out.ridge.cv.bh, main = "Linear Regression Model with Ridge Penalty")
out.ridge.cv.bh$lambda.min

coef(out.ridge.cv.bh, s = "lambda.min")

```

```

final.ridge.model.bh <- glmnet(x = Xtrain.bh.perm.lasso, y = ytrain,
                             family = "binomial",
                             lambda = out.ridge.cv.bh$lambda.min, alpha = 0)

yhat.ridge.cv.bh <- predict(final.ridge.model.bh, newx = Xtest[, lasso.genes.bh],
                           type= "response", s = "lambda.min")
yhat.ridge.cv.bh

yhat.ridge.cv.bh.discrete <- ifelse(yhat.ridge.cv.bh > 0.5, "control", "cancer")
View(cbind(as.factor(yhat.ridge.cv.bh.discrete), ytest))

confusion.glmnet(final.ridge.model.bh,
                 newx = Xtest[, lasso.genes.bh], newy = ytest,
                 family = "binomial") -> confusion_bh
confusion_bh

write.table(confusion_bh , file = "immagini_tabelle\\4d_confusionM_bh.csv")

as.matrix(coef(final.ridge.model.bh)) %>%
  row.names()

tibble(Coeff = as.matrix(coef(final.ridge.model.bh))[,1]) -> coeff
coeff["Gene Name"] = as.matrix(coef(final.ridge.model.bh)) %>%
  row.names()

write.csv(coeff, file="immagini_tabelle\\5_final_model_coeff.csv")

coeff <- coeff[-1,]
coeff %>% arrange(desc(Coeff)) -> coeff
as.matrix(coeff$Coeff)
barplot(t(as.matrix(coeff$Coeff)), las=2, ylab = "abs(Coefficient)")

# single boxplots
for (gene in lasso.genes.bh) {
  png(filename = paste("immagini_tabelle\\boxplots\\", gene, ".png", sep = ""))
  boxplot(X[, gene] ~ y, main= gene, ylab = "Gene Expression Level",
          xlab = "")
  dev.off()
}

# first graph
par(mfrow=c(2,7))
for (gene in lasso.genes.bh[1:14]) {
  boxplot(X[, gene] ~ y, main= gene, ylab = "Gene Expression Level",
          xlab = "")
}

# second graph
par(mfrow=c(2,7))
for (gene in lasso.genes.bh[15:27]) {
  boxplot(X[, gene] ~ y, main= gene, ylab = "Gene Expression Level",
          xlab = "")
}

```


Results

The calculation of the significant genes by t-test between mean of control and cancer samples on **theoretical** and **permutation distributions**, allowed to obtain p-values for each of the 6033 genes in the dataset.

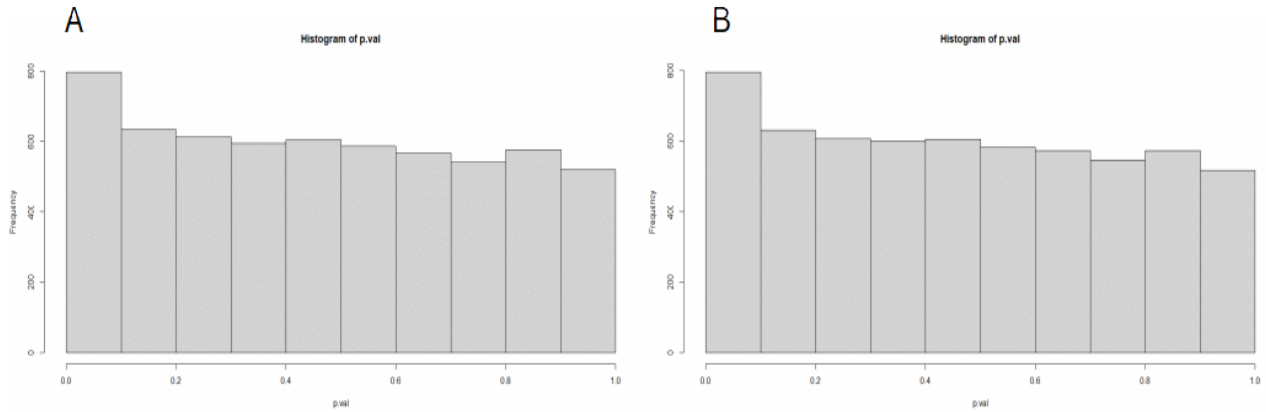


Figure 1: p-values distributions obtained through margin T-Test by (A) Theoretical and (B) Permutation methods.

Starting from the p-values of the t-test procedures, **MTPs** were performed, and it was obtained a higher number of null hypothesis rejected with permutation method with respect to theoretical one. Obviously permutation method, require a lot of computation since it was needed about 4-5 hours to compute the calculations with B (number of permutation per gene) equal to 5000. As the opposite, theoretical procedure required only few minutes.

MTP on Theoretical Distribution			MTP by Permutation Method		
Procedure	NoRej_H0	Rej_H0	Procedure	NoRej_H0	Rej_H0
Decision .05	5555	478	Decision .05	5552	481
Bonferroni	6031	2	Bonferroni	6008	25
SidakSS	6031	2	SidakSS	6008	25
Holm	6031	2	Holm	6008	25
SidakSD	6031	2	SidakSD	6008	25
Hochberg	6031	2	Hochberg	6008	25
BH	6012	21	BH	5998	35
BY	6031	2	BY	6008	25

Table1: Summary of the two procedures by (on the left side) theoretical and (on the right side) permutation methods. For each row are shown number of rejected and not rejected null hypothesis.

Since we are dealing with biological data, we are not well aware about the null distribution and we preferred to use only permutation method.

In particular, the subsequent analysis were performed on the 481 significant genes with $p < 0.05$ without the correction (1st path) and on the 35 significant genes with BH corrected p-values with $p < 0.05$ (2nd path). Lasso Regressions after the tuning of lambda parameters to minimize MSE by CV, gave back a selection of 51 and 27 genes without and with MTP-BH correction, respectively. Lasso Selection thus, pushed to zero several coefficients.

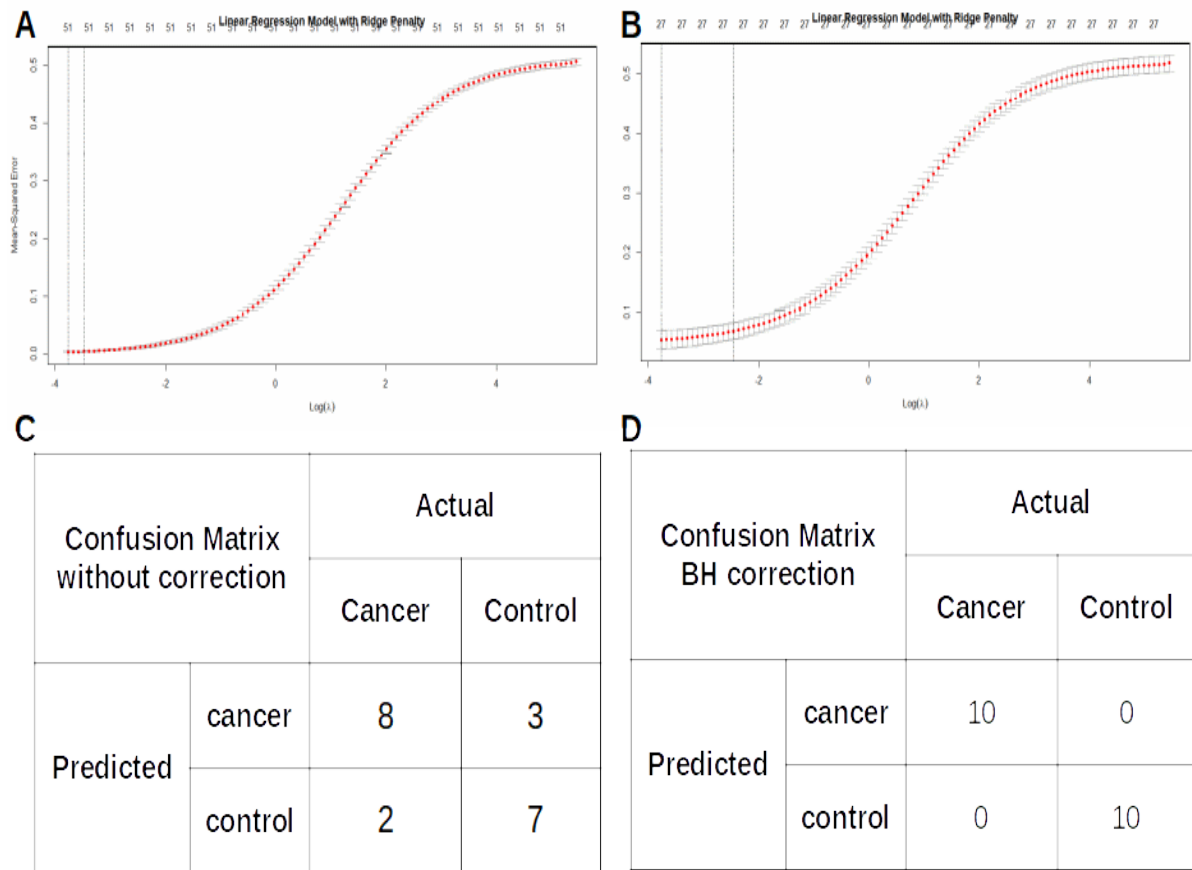


Figure 3: CV for Ridge Regression. First Row: Validation Curves on Train dataset by CV of MSE in function of the lambda parameter in model (A) with and (B) without BH correction. Second row: Confusion matrix of the final classifier on the test set.

The Ridge penalization pushed several coefficients toward lower values, **regularizing** the final model which showed better **generalization** capacity.

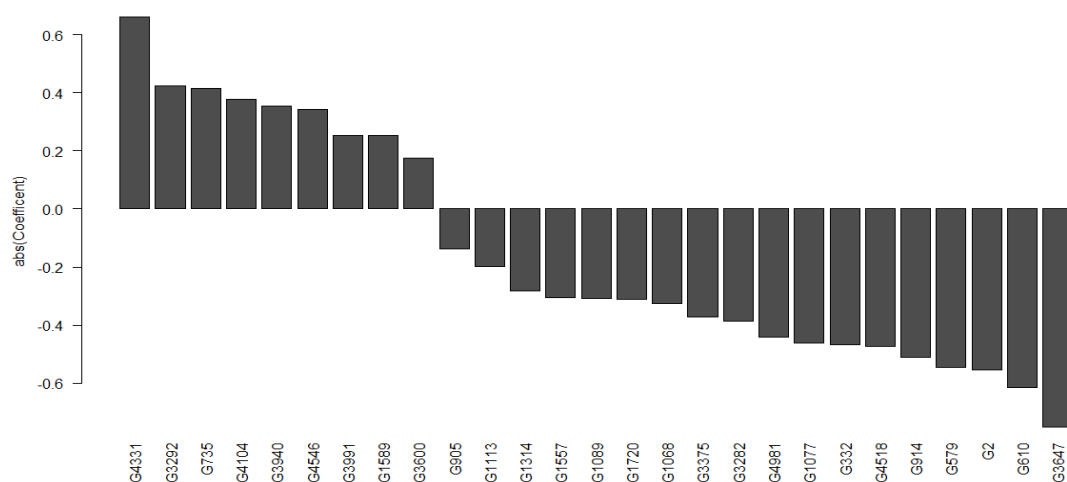


Figure 4: Coefficients of the Final Logistic Regression model built through the second path with a preliminar BH correction of p-values before Lasso and Ridge Regression steps.

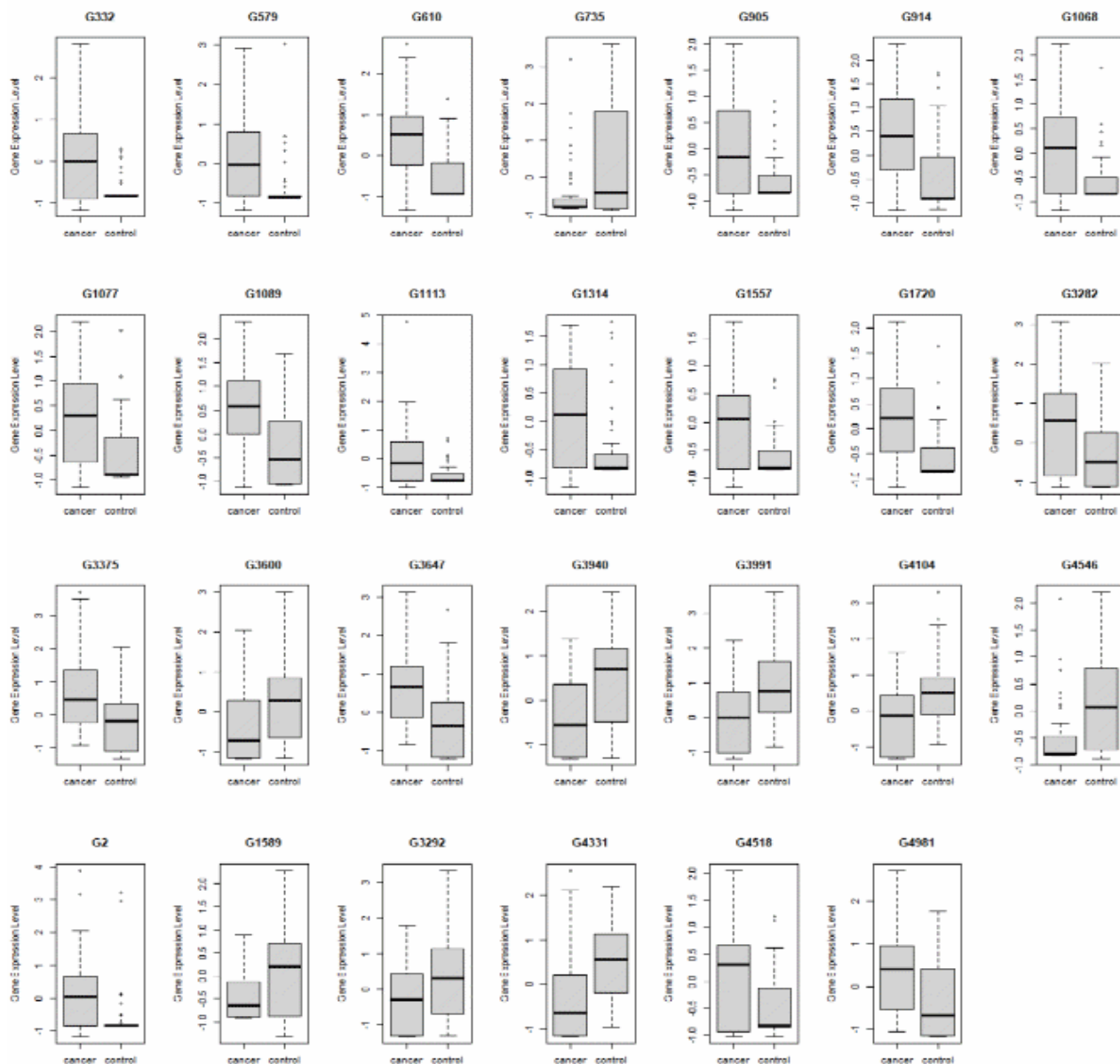


Figure 5: Box Plots of genes used by the final best model.

Discussion

The final Ridge Logistic Regression model built through the use of the genes screened firstly by MTP with BH's correction (35 significant expression levels) and then by Cross Validated Lasso Logistic Regression's features selection (27 genes), gave back an accuracy of 100%.

The same model, built without the MTP screening of the genes (481 significant genes' expression levels) was less accurate, with an overall accuracy of 75%, misclassifying 3 control and 2 cancer samples, showing thus, overfitting on training data with respect the less complicated model, trained on a minor number of features, that over-performed the first one.

Summarizing, the most important step in this procedures was, probably, the screening made by MTP since it was able to improve strongly the final performance of the model on new unseen data. It was obtained a good Logistic Regression model able to generalize well the classification tasks.